

Classical Machine Learning

Week 0

Plan

- Setting up your learning and programming environment

Getting started

- [Setting up your ML environment \(Setup_NYU.ipynb\)](#)
 - [Choosing an ML environment](#)
[\(Choosing an ML Environment NYU.ipynb\)](#)
- [Quick intro to the tools \(Getting_Started.ipynb\)](#)

Week 1

Plan

We give a brief introduction to the course.

We then present the key concepts that form the basis for this course

- For some: this will be review

Intro to Advanced Course

- [Introduction to Advanced Course \(Intro_Advanced.ipynb\)](#)

Here is a *quick reference* of key concepts/notations from the Intro course

- For some: it will be a review, for others: it will be a preview.
- We will devote a sub-module of this lecture to elaborate on each topic in slightly more depth.
 - For a more detailed explanation: please refer to the material from the Intro course ([repo \(https://github.com/kenperry-public/ML_Spring_2023\)](https://github.com/kenperry-public/ML_Spring_2023))

Review/Preview of concepts from Intro Course

Transformers: Review

Preview

There is lots of interest in Large Language Models (e.g., ChatGPT). These are based on an architecture called the Transformer. We will introduce the Transformer and demonstrate some amazing results achieved by using Transformers to create Large Language Models.

Attention is a mechanism that is a core part of the Transformer. We will begin by first introducing Attention.

We will then take a detour and study the Functional model architecture of Keras. Unlike the Sequential model, which is an ordered sequence of Layers, the organization of blocks in a Functional model is more general. The Advanced architectures (e.g., the Transformer) are built using the Functional model.

Once we understand the technical prerequisites, we will examine the code for the Transformer.

[Transformers: Review \(Review Transformer.ipynb\)](#)

Suggested reading

- Attention
 - [Attention is all you need \(https://arxiv.org/pdf/1706.03762.pdf\)](https://arxiv.org/pdf/1706.03762.pdf)
- Transfer Learning
 - [Sebastian Ruder: Transfer Learning \(https://ruder.io/transfer-learning/\)](https://ruder.io/transfer-learning/)

Further reading

- Attention
 - [Neural Machine Translation by Jointly Learning To Align and Translate \(https://arxiv.org/pdf/1409.0473.pdf\)](https://arxiv.org/pdf/1409.0473.pdf)
 - Geron Chapter 16

Week 2: Review/Preview (continued); Technical

Plan

We continue the review/preview of key concepts that we started last week.

Our ultimate goal is to introduce the Transformer (which uses Attention heavily) in theory, and demonstrate its use in Large Language Models.

Review/preview continued

[Transformer motivation: Illustrated \(Attention_motivation_illustrated.ipynb\)](#)

[Transformer: flavors \(Transformer.ipynb#Transformer-variants\)](#)

Attention: in depth

- [Implementing Attention \(Attention_Lookup.ipynb\)](#)

Transfer Learning: Review (Review_TransferLearning.ipynb)

-- -- -- -- -- -- -- --

Functional Models

Plan

Enough theory (for the moment) !

The Transformer (whose theory we have presented) is built from plain Keras.

Our goal is to dig into the **code** for the Transformer so that you too will learn how to build advanced models.

Before we can do this, we must

- go beyond the Sequential model of Keras: introduction to the Functional model
- understand more "advanced" features of Keras: customomizing layers, training loops, loss functions
- The Datasets API

Basics

We start with the basics of Functional models, and will give a coding example of such a model in Finance.

- [Functional API \(Functional Models.ipynb\)](#)

Functional Model Code: A Functional model in Finance: "Factor model"

We illustrate the basic features of Functional models with an example

- does not use the additional techniques of the next section (Advanced Keras)

[Autoencoders for Conditional Risk Factors](#)

[\(Autoencoder for conditional risk factors.ipynb\)](#)

- [code \(https://github.com/stefan-jansen/machine-learning-for-trading/blob/main/20_autoencoders_for_conditional_risk_factors/06_conditional a](https://github.com/stefan-jansen/machine-learning-for-trading/blob/main/20_autoencoders_for_conditional_risk_factors/06_conditional_a)

Additional Deep Learning resources

Here are some resources that I have found very useful.

Some of them are very nitty-gritty, deep-in-the-weeds (even the "introductory" courses)

- For example: let's make believe PyTorch (or Keras/TensorFlow) didn't exist; let's invent Deep Learning without it !
 - You will gain a deeper appreciation and understanding by re-inventing that which you take for granted

[Andrej Karpathy course: Neural Networks, Zero to Hero \(https://karpathy.ai/zero-to-hero.html\)](https://karpathy.ai/zero-to-hero.html)

- PyTorch
- Introductory, but at a very deep level of understanding
 - you will get very deep into the weeds (hand-coding gradients !) but develop a deeper appreciation

fast.ai

`fast.ai` is a web-site with free courses from Jeremy Howard.

- PyTorch
- Introductory and courses "for coders"
- Same courses offered every few years, but sufficiently different so as to make it worthwhile to repeat the course !
 - [Practical Deep Learning](https://course.fast.ai/) (<https://course.fast.ai/>)
 - [Stable diffusion](https://course.fast.ai/Lessons/part2.html) (<https://course.fast.ai/Lessons/part2.html>)
 - Very detailed, nitty-gritty details (like Karpathy) that will give you a deeper appreciation

Stefan Jansen: Machine Learning for Trading (<https://github.com/stefan-jansen/machine-learning-for-trading>)

An excellent github repo with notebooks

- using Deep Learning for trading
- Keras
- many notebooks are cleaner implementations of published models

Assignments

Your assignments should follow the [Assignment Guidelines](#)
([assignments/Assignment_Guidelines.ipynb](#)).

```
In [1]: print("Done")
```

Done

